

ACM-ICPC TEAM REFERENCE DOCUMENT

Skill++ (Gogolev, Drachev)

Contents

1	General	1
1.1	C++ Template	1
1.2	Compilation	1
1.3	Check	1
1.4	How to solve problems	2
2	Math	2
2.1	Simpson	2
3	Strings	2
4	Geometry	2
4.1	Manhatan Spanning	2

1 General

1.1 C++ Template

```
#pragma GCC optimize("O3,unroll-loops")
#define _CRT_SECURE_NO_WARNINGS
#define ln '\n'
#define All(x) (x).begin(), (x).end()
#define Allr(x) (x).rbegin(), (x).rend()
#define Allf(x) x.begin() + 1, x.end()
#define fi(a, b) for (auto i = (a); i <= (b); i++)
#define fj(a, b) for (auto j = (a); j <= (b); j++)
#define fo(a, b) for (auto o = (a); o <= (b); o++)
#define fq(a, b) for (auto q = (a); q <= (b); q++)
#define fe(a, b) for (auto e = (a); e <= (b); e++)
#define fw(a, b) for (auto w = (a); w <= (b); w++)
#define fdi(a, b) for (auto i = (a); i >= (b); i--)
#define fdj(a, b) for (auto j = (a); j >= (b); j--)
#define fdo(a, b) for (auto o = (a); o >= (b); o--)
#define fdq(a, b) for (auto q = (a); q >= (b); q--)
#define fde(a, b) for (auto e = (a); e >= (b); e--)
#define fdw(a, b) for (auto w = (a); w >= (b); w--)
#define fx(A) for (auto &x : (A))
#define fy(A) for (auto &y : (A))
#define fz(A) for (auto &z : (A))
#define pb push_back
#define mp make_pair
#define ins insert
#define sz(x) (int)x.size()
#include <string>
#include <vector>
#include <list>
#include <map>
#include <set>
#include <deque>
#include <queue>
#include <stack>
#include <bitset>
#include <algorithm>
#include <sstream>
#include <iostream>
#include <iomanip>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <cstring>
#include <cmath>
#include <cstdlib>
#include <cassert>
#include <ctime>
#include <tuple>
#include <unordered_set>
#include <unordered_map>
#include <random>
```

```
#include <chrono>
#include <complex>
using namespace std;
#ifdef _DEBUG
#define LOCAL
#endif
mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch
().count());
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef vector<bool> vb;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<ld> vd;
typedef vector<char> vc;
typedef vector<string> vs;
typedef vector<vb> vvb;
typedef vector<vi> vvi;
typedef vector<vll> vvll;
typedef vector<vc> vvc;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
template<typename A, typename B>
ostream& operator<<(ostream& os, pair<A, B> p) {
    os << "(" << p.first << ", " << p.second << ")";
    return os;
}
template<typename T>
ostream& operator<<(ostream& os, vector<T> v) {
    fi(0, sz(v) - 1) {
        os << v[i] << " ";
    }
    return os;
}
template<typename T>
ostream& operator<<(ostream& os, set<T> t) {
    for (auto z : t) {
        os << z << " ";
    }
    return os;
}
template<typename T1, typename T2>
ostream& operator<<(ostream& os, map<T1, T2> t) {
    cerr << endl;
    for (auto z : t) {
        os << "\t" << z.first << " -> " << z.second << endl;
    }
    return os;
}
#ifdef LOCAL
#define dbg(x) {cerr << __LINE__ << "\t" << #x << " : " << (x) << endl;}
#else
#define dbg(x) {}
#endif
#define LOCAL
#define ass(x) if (!x) { cerr << __LINE__ << "\tassertion_
failed: " << #x << endl; abort(); }
#else
#define ass(x) assert(x)
#endif
void solve() {

}
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    cout << setprecision(20);
    #ifdef LOCAL
        freopen("input.txt", "r", stdin);
        freopen("output.txt", "w", stdout);
    #endif
    solve();
}
```

1.2 Compilation

```
#far association
g++ --std=c++17 -DLOCAL -O2 -Wall -Wl,--stack
=67108864 -o !.exe
```

1.3 Check

```
:start
gen.exe
sol.exe
copy output.txt answer.txt
NAME.exe
fc output.txt answer.txt
if ERRORLEVEL 1 goto end
goto start
:end
```

1.4 How to solve problems

1. Math formulation
2. Using parameters
3. Go to theory
4. Way od solutions:
 - (a) fix parameter
 - (b) cases
 - (c) see on answer
 - (d) induction
 - (e) hypothesis
 - (f) finding of
 - (g) simplification
 - (h) reduction to other problem
 - (i) reduction to classic problem
 - (j) reduction of classic problem to our
 - (k) modeling
 - (l) generalization
 - (m) contrexamples
 - (n) full random
 - (o) invariant

2 Math

2.1 Simpson

```
double a = 0, b = 1;
const int n = 100000000;
double s = 0;
double h = (b - a) / n;
fi(0, n) {
    double x = a + h * i;
    s += fun(x) * ((i==0 || i==n) ? 1 : ((i&1)==0) ? 2 : 4);
}
s *= h / 3;
```

2.2 Prime

```
ll fun1(ll a, ll n) {
    ll t = n - 1;
    int q = 0;
    while(t % 2 == 0) {
        q++;
        t /= 2;
    }
    ll v = pown(a, t, n);
    fi(1, q) {
        ll w = mult(v, v, n);
        if(v != 1 && v != n - 1 && w == 1) return true;
    }
```

```
        v = w;
    }
    if(v > 1) return true;
    return false;
}

bool is_prime(ll n) {
    if(n < 2) return false;
    if(n == 2) return true;
    if(n % 2 == 0) return false;
    fi(1, 10) {
        ll a = next(1, n - 1);
        if(fun1(a, n)) return false;
    }
    return true;
}
```

3 Strings

4 Geometry

4.1 Manhatan Spanning

```
// Returns a list of edges in the format (weight, u, v).
// Passing this list to Kruskal algorithm will give the
// Manhattan MST.
vector<tuple<long long, int, int>> manhattan_mst_edges(
    vector<point> ps) {
    vector<int> ids(ps.size());
    iota(ids.begin(), ids.end(), 0);
    vector<tuple<long long, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) { // for every rotation
        sort(ids.begin(), ids.end(), [&](int i, int j){
            return (ps[i].x + ps[i].y) < (ps[j].x + ps[j].y);
        });
        map<int, int, greater<int>> active; // (xs, id)
        for (auto i : ids) {
            for (auto it = active.lower_bound(ps[i].x); it !=
                active.end(); it++) {
                active.erase(it++);
                int j = it->second;
                if (ps[i].x - ps[i].y > ps[j].x - ps[j].y) break;
                assert(ps[i].x >= ps[j].x && ps[i].y >= ps[j].y);
                edges.push_back({(ps[i].x - ps[j].x) + (ps[i].y - ps[
                    j].y), i, j});
            }
            active[ps[i].x] = i;
        }
        for (auto &p : ps) { // rotate
            if (rot & 1) p.x *= -1;
            else swap(p.x, p.y);
        }
    }
    return edges;
}
```